BASES DE DATOS II

Triggers en FloatMate

Guia entendible de validacion automatica y auditoria

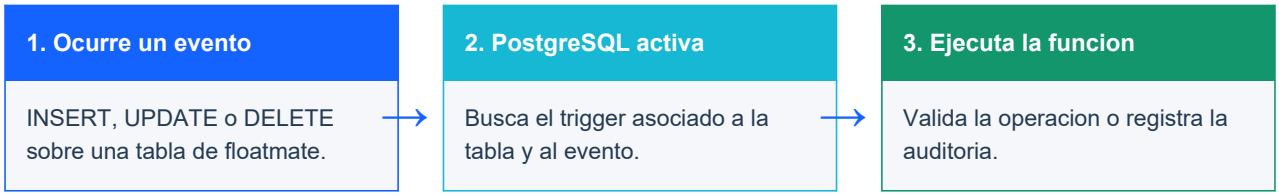
11 triggers	2 de validacion	9 de auditoria

Nodo Norte - esquema floatmate_programacion
Preparado para estudio y sustentacion

1. La idea en palabras sencillas

Un trigger es una regla automatica que PostgreSQL ejecuta cuando alguien inserta, actualiza o elimina una fila. No se llama manualmente: la base lo activa al detectar el evento.

> **En FloatMate los triggers cumplen dos trabajos: impedir viajes invalidos y conservar evidencia de los cambios.**



¿Donde estan?

Los triggers estan conectados a las tablas fisicas de Norte → floatmate_norte → floatmate. Las funciones que ejecutan estan dentro de floatmate_programacion.

> **Importante: esta implementacion instala los triggers en el nodo Norte. Los cambios realizados directamente sobre las tablas fisicas de Sur no activan la auditoria de Norte.**

Como se construye un trigger

```
CREATE FUNCTION floatmate_programacion.mi_funcion_trigger()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    -- Logica automatica
    RETURN NEW;
END;
$$;

CREATE TRIGGER mi_trigger
BEFORE INSERT OR UPDATE ON floatmate.mi_tabla
FOR EACH ROW
EXECUTE FUNCTION floatmate_programacion.mi_funcion_trigger();
```

2. Inventario de los 11 triggers

Trigger	Tabla	Momento	Funcion
tr_validar_viaje	viajes	BEFORE	Capacidad y terminal
tr_validar_inicio_viaje	viajes	BEFORE	Control antes de salir
tr_auditar_usuarios	usuarios	AFTER	Auditoria
tr_auditar_conductores	conductores	AFTER	Auditoria
tr_auditar_vehiculos	vehiculos	AFTER	Auditoria
tr_auditar_rutas	rutas	AFTER	Auditoria
tr_auditar_viajes	viajes	AFTER	Auditoria
tr_auditar_controles	controles_salida	AFTER	Auditoria
tr_auditar_abastecimientos	abastecimientos	AFTER	Auditoria
tr_auditar_mantenimientos	mantenimientos	AFTER	Auditoria
tr_auditar_incidentes	incidentes	AFTER	Auditoria

Resumen: 2 triggers protegen las reglas del negocio y 9 triggers dejan trazabilidad.

3. Trigger tr_validar_viaje

Se ejecuta antes de insertar un viaje o de modificar sus recursos y pasajeros. Su funcion es impedir que llegue informacion incoherente a la tabla.

Validaciones realizadas

Regla	Que evita
El vehiculo debe existir	Viajes asociados a vehiculos inexistentes
El conductor debe existir	Asignaciones sin conductor real
La ruta debe existir	Viajes sin una ruta valida
Todos deben ser de la misma terminal	Mezclar recursos NOR y SUR
Pasajeros <= capacidad	Sobrecupo del vehiculo

Ejemplo rechazado

```
INSERT INTO floatmate.viajes (
    terminal_codigo, vehiculo_id, conductor_id, ruta_id,
    fecha_hora_programada, pasajeros, estado
)
VALUES ('NOR', 1001, 1001, 1001, CURRENT_TIMESTAMP, 99, 'PROGRAMADO');
```

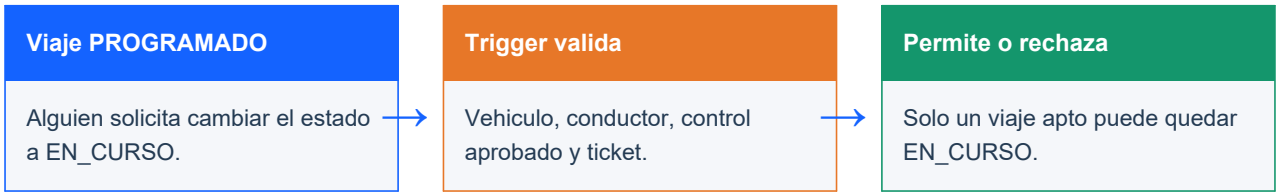
Resultado: el trigger ejecuta RAISE EXCEPTION porque 99 pasajeros superan la capacidad. La fila no se guarda.

¿Por que es BEFORE?

Porque debe revisar la fila antes de almacenarla. Si la regla falla, la excepcion cancela el INSERT o UPDATE completo.

4. Trigger tr_validar_inicio_viaje

Se activa antes de cambiar el estado de un viaje a EN_CURSO. Su objetivo es que ningun bus salga sin cumplir los controles operativos.



Condiciones revisadas

- El viaje debe existir y estar en un estado adecuado.
- El vehiculo debe estar disponible.
- El conductor debe estar disponible.
- Debe existir un control de salida.
- El control debe tener resultado APROBADO.
- Debe existir un numero de ticket.

Ejemplo

```
UPDATE floatmate.viajes
SET estado = 'EN_CURSO'
WHERE viaje_id = 1001;
```

> **Si falta el control aprobado, PostgreSQL responde: No se puede iniciar el viaje: falta un control de salida aprobado.**

Valor para el negocio

La regla no depende de que una pantalla recuerde validarla. Incluso si alguien ejecuta un UPDATE directo, la propia base protege la operacion.

5. Los 9 triggers de auditoria

Cada tabla principal tiene un trigger AFTER. Todos reutilizan la misma funcion generica: fn_trg_auditar_cambios().

Evento	datos_anteriores	datos_nuevos
INSERT	NULL	Fila insertada
UPDATE	Fila antes del cambio	Fila despues del cambio
DELETE	Fila eliminada	NULL

Informacion registrada

Fecha y hora, usuario de PostgreSQL, operacion, esquema, tabla, clave del registro, datos anteriores, datos nuevos y direccion del cliente.

¿Por que AFTER?

Porque la auditoria se registra despues de que la operacion principal fue aceptada. Si la transaccion completa termina en ROLLBACK, la auditoria tambien se revierte.

¿Por que una sola funcion para nueve tablas?

Cada trigger envia como argumento el nombre de su clave primaria. Por ejemplo, vehiculos envia 'vehiculo_id'. La funcion lee ese valor con TG_ARGV[0] y puede identificar la fila sin conocer previamente la estructura de la tabla.

```
CREATE TRIGGER tr_auditar_vehiculos
AFTER INSERT OR UPDATE OR DELETE ON floatmate.vehiculos
FOR EACH ROW
EXECUTE FUNCTION
    floatmate_programacion.fn_trg_auditar_cambios('vehiculo_id');
```

6. Variables especiales y JSONB

Variable	Significado	Se usa en
NEW	Fila nueva	INSERT y UPDATE
OLD	Fila anterior	UPDATE y DELETE
TG_OP	Operacion que activo el trigger	INSERT, UPDATE o DELETE
TG_TABLE_NAME	Nombre de la tabla	Auditoria generica
TG_TABLE_SCHEMA	Esquema de la tabla	Auditoria generica
TG_ARGV[0]	Primer argumento del trigger	Nombre de la clave primaria

¿Por que JSONB?

Las nueve tablas tienen columnas distintas. JSONB permite guardar una representacion completa de cualquier fila sin crear una tabla de auditoria diferente para cada entidad.

```
{
  "vehiculo_id": 1001,
  "placa": "FTA101",
  "estado": "DISPONIBLE",
  "kilometraje": 46750
}
```

> **En un UPDATE se guardan dos documentos JSONB: uno con OLD y otro con NEW. Asi se puede saber exactamente que cambio.**

7. Como consultar y probar los triggers

Todas estas consultas se ejecutan desde Norte → floatmate_norte.

Listar triggers

```
SELECT event_object_table AS tabla,
       trigger_name,
       event_manipulation AS evento,
       action_timing AS momento
FROM information_schema.triggers
WHERE action_statement ILIKE '%floatmate_programacion%'
ORDER BY tabla, trigger_name, evento;
```

Contar triggers reales

```
SELECT count(DISTINCT trigger_name) AS total_triggers
FROM information_schema.triggers
WHERE action_statement ILIKE '%floatmate_programacion%';
```

Resultado esperado: 11.

Consultar auditoria

```
SELECT auditoria_id, fecha_hora, usuario_bd, operacion,
       nombre_tabla, clave_registro,
       datos_anteriores, datos_nuevos
FROM floatmate_programacion.auditoria_cambios
ORDER BY auditoria_id DESC;
```

8. Demostracion segura y respuesta para el profesor

Prueba de auditoria sin dejar cambios

```
BEGIN;

UPDATE floatmate.vehiculos
SET estado = 'MANTENIMIENTO'
WHERE vehiculo_id = 1001;

SELECT operacion, nombre_tabla, clave_registro,
       datos_anteriores, datos_nuevos
FROM floatmate_programacion.auditoria_cambios
WHERE nombre_tabla = 'vehiculos'
ORDER BY auditoria_id DESC
LIMIT 1;

ROLLBACK;
```

El UPDATE activa la auditoria y permite verla dentro de la transaccion. ROLLBACK revierte tanto el cambio como su auditoria, por lo que los datos de la entrega quedan intactos.

Respuesta corta para memorizar

> **Implementamos once triggers en el nodo Norte. Dos son triggers BEFORE que validan la capacidad, la coherencia de terminal y el control requerido para iniciar un viaje. Los otros nueve son triggers AFTER de auditoria, uno por tabla. Todos reutilizan una funcion generica que almacena en JSONB los valores anteriores y nuevos de cada modificacion.**

Pregunta dificil: ¿los triggers funcionan en Sur?

Respuesta honesta: en la implementacion actual estan instalados en las tablas fisicas de Norte. Para auditar cambios directos sobre Sur, se instalaria el mismo modulo adaptado en floatmate_sur. Las tablas extranjeras de Norte no reemplazan los triggers del nodo remoto.

Referencia

PostgreSQL - CREATE TRIGGER: https://www.postgresql.org/docs/current/sql-createtrigger.html
PostgreSQL - Trigger Functions: https://www.postgresql.org/docs/current/plpgsql-trigger.html